1. Install & Configure Riak KV (Student Bucket CRUD)

Install Riak KV on Linux. Create bucket `students` and perform basic HTTP Key-Value operations (PUT, GET, DELETE).

TERMINAL / HTTP COMMANDS:

```
# Install Riak KV & Start Daemon
sudo apt-get install riak && sudo riak start

# 1. STORE (PUT) Student Record
curl -i -X PUT http://localhost:8098/buckets/students/keys/stu101 \
  -H "Content-Type: application/json" \
  -d '{"name": "Aarav Sharma", "course": "BCA", "cgpa": 8.5}'

# 2. RETRIEVE (GET) Student Record
curl -i -X GET http://localhost:8098/buckets/students/keys/stu101

# 3. DELETE Student Record
curl -i -X DELETE http://localhost:8098/buckets/students/keys/stu101
```

HTTP RESPONSE OUTPUT:

```
HTTP/1.1 204 No Content
VClock: a85hY2Bmc2BGMnBlYwISYIAxMIGCYPBlwAwyC2VpYgA=

HTTP/1.1 200 OK
Content-Type: application/json
{"name": "Aarav Sharma", "course": "BCA", "cgpa": 8.5}
```

2. User Sessions Bucket in Riak KV

Create `user_sessions` bucket storing session_id, user_id, login time, and status. Retrieve by session key.

RIAK KV COMMANDS:

```
# Store User Session Data
curl -X PUT http://localhost:8098/buckets/user_sessions/keys/sess_9901 \
  -H "Content-Type: application/json" \
  -d '{"user_id": "USR_55", "login_time": "2026-10-31T09:00:00Z", "status": "active"}'

# Fetch Session Information
curl -X GET http://localhost:8098/buckets/user_sessions/keys/sess_9901
```

EXECUTION OUTPUT:

```
HTTP/1.1 200 OK
Content-Type: application/json
{"user_id": "USR_55", "login_time": "2026-10-31T09:00:00Z", "status": "active"}
```

3. User Profiles & Preferences Store

Store user preferences (theme, language, notifications) in bucket `user_profiles` with JSON payloads.

RIAK KV COMMANDS:

```
curl -X PUT http://localhost:8098/buckets/user_profiles/keys/user_aarav \
  -H "Content-Type: application/json" \
  -d '{"name":"Aarav","email":"aarav@coer.ac.in","city":"Roorkee","prefs":
{"theme":"dark","lang":"en"}}'

curl -X GET http://localhost:8098/buckets/user_profiles/keys/user_aarav
```

EXECUTION OUTPUT:

```
HTTP/1.1 200 OK
{"name":"Aarav","email":"aarav@coer.ac.in","city":"Roorkee","prefs":{"theme":"dark","lang":"en"}}
```

4. Riak Shopping Cart Application

Manage cart sub-items inside `carts` bucket. Update cart content using GET-modify-PUT pattern.

RIAK KV COMMANDS:

```
# Add/Update Cart Item
curl -X PUT http://localhost:8098/buckets/carts/keys/cart_user88 \
  -H "Content-Type: application/json" \
  -d '{"items": [{"product_id":"P10","qty":2,"price":499}], "total": 998}'

curl -X GET http://localhost:8098/buckets/carts/keys/cart_user88
```

EXECUTION OUTPUT:

```
HTTP/1.1 200 OK
{"items": [{"product_id":"P10","qty":2,"price":499}], "total": 998}
```

5. Data Types in Key-Value Stores

Store raw text strings, JSON documents, and binary lists in Riak buckets to demonstrate schema-agnostic storage.

RIAK KV COMMANDS:

```
# 1. Plain String
curl -X PUT http://localhost:8098/buckets/data/keys/str1 -H "Content-Type: text/plain" -d "Hello
Riak"

# 2. JSON Document
curl -X PUT http://localhost:8098/buckets/data/keys/json1 -H "Content-Type: application/json" -d
'{"key":"val"}'

# 3. List Data
curl -X PUT http://localhost:8098/buckets/data/keys/list1 -H "Content-Type: application/json" -d
'[1,2,3,4]'
```

EXECUTION OUTPUT:

```
Role Explanation:
- Bucket: Logical namespace grouping keys.
- Key: Unique string locator within a bucket.
- Value: Opaque binary payload (JSON, Text, Image) associated with a key.
```

6. Tunable Quorums (N, R, W) for Consistency

Configure Read (R) and Write (W) quorums in Riak queries to adjust CAP theorem trade-offs between consistency and speed.

RIAK KV COMMANDS:

```
# Read with R=2 (Quorum Read across replicas)
curl -X GET "http://localhost:8098/buckets/students/keys/stu101?r=2"

# Write with W=all (Strict Consistency write)
curl -X PUT "http://localhost:8098/buckets/students/keys/stu101?w=all" \
  -H "Content-Type: application/json" -d '{"name":"Aarav","status":"updated"}'
```

EXECUTION OUTPUT:

```
HTTP/1.1 200 OK
Note: N=3 (Replication factor), R=2, W=2 guarantees strong read-your-writes consistency.
```

7. Data Partitioning & Horizontal Scaling

Demonstrate how Riak uses consistent hashing ring (160-bit space) to partition data automatically across cluster nodes.

RIAK ADMIN SHELL:

```
# Check cluster node status & ring ownership
riak-admin cluster status
riak-admin ring-status
```

EXECUTION OUTPUT:

```
================================== Membership ==================================
Status     Ring    Pending    Node
--------------------------------------------------------------------------------
valid     33.3%      --       'dev1@127.0.0.1'
valid     33.3%      --       'dev2@127.0.0.1'
valid     33.4%      --       'dev3@127.0.0.1'
Key distribution occurs via SHA-1 hashing of (Bucket + Key).
```

8. Multi-Operation Scenarios & Transaction Limitations

Execute multi-step cart operations in Riak and analyze limitations of key-value stores without multi-key ACID support.

RIAK VECTOR CLOCK WORKFLOW:

```
# Read item to obtain VClock header
curl -i http://localhost:8098/buckets/carts/keys/cart_88

# Perform conditional update passing X-Riak-VClock header
curl -X PUT http://localhost:8098/buckets/carts/keys/cart_88 \
  -H "X-Riak-VClock: a85hY2Bmc2BGMnBlYwISYIAxMIGCYPBlwAwyC2VpYgA=" \
  -H "Content-Type: application/json" -d '{"items":["P1","P2"],"total":1200}'
```

LIMITATIONS SUMMARY:

```
Riak provides eventual consistency per key. Multi-key ACID transactions are unsupported;
application logic must handle concurrent conflicts via Vector Clocks or CRDTs.
```

9. Query-by-Data using Secondary Indexes (2i)

Attach secondary indexes to Riak keys to query user records by city or age attributes without full bucket scans.

RIAK 2I COMMANDS:

```
# Store record with Secondary Index header (x-riak-index-<name>_bin)
curl -X PUT http://localhost:8098/buckets/users/keys/usr_101 \
  -H "x-riak-index-city_bin: Roorkee" \
  -H "x-riak-index-age_int: 21" \
  -H "Content-Type: application/json" \
  -d '{"name":"Aarav Sharma","city":"Roorkee","age":21}'

# Query Index for users in Roorkee
curl http://localhost:8098/buckets/users/index/city_bin/Roorkee
```

EXECUTION OUTPUT:

```
{"keys":["usr_101"]}
```

10. Riak Data Types (CRDT Sets)

Use Conflict-Free Replicated Data Types (CRDT Sets) to perform atomic add, remove, and membership operations on user interests.

RIAK CRDT COMMANDS:

```
# Add elements to CRDT Set 'interests'
curl -X POST http://localhost:8098/types/sets/buckets/user_interests/keys/usr_101 \
  -H "Content-Type: application/json" \
  -d '{"add_all": ["Coding", "Robotics", "Cricket"]}'

# Fetch CRDT Set
curl http://localhost:8098/types/sets/buckets/user_interests/keys/usr_101
```

EXECUTION OUTPUT:

```
{"type":"set","value":["Coding","Cricket","Robotics"]}
```

11. Install Neo4j & Create Academic Graph

Install Neo4j DBMS. Create `Student`, `Course`, and `Department` nodes connected by relationships.

CYPHER SCRIPT:

```
CREATE (d:Department {name: "Computer Applications"})
CREATE (c:Course {code: "BCA305P", title: "NoSQL Database Design"})
CREATE (s:Student {student_id: "STU101", name: "Aarav Sharma"})

CREATE (s)-[:ENROLLED_IN]->(c)
CREATE (c)-[:OFFERED_BY]->(d)
RETURN s, c, d;
```

CYPHER OUTPUT:

```
+-------------------------------------------------------------------------+
| s                              | c                        | d           |
+-------------------------------------------------------------------------+
| (:Student {name: "Aarav Sharma"})| (:Course {code:"BCA305P"})| (:Dept)   |
+-------------------------------------------------------------------------+
Created 3 nodes, created 2 relationships in 24ms.
```

12. Social Network Graph in Neo4j Cypher

Create a network of `Person` nodes with `FRIENDS_WITH` and `FOLLOWS` edges. Query friends and followers.

CYPHER SCRIPT:

```
CREATE (p1:Person {name: "Aarav"}), (p2:Person {name: "Riya"}), (p3:Person {name: "Kabir"})
CREATE (p1)-[:FRIENDS_WITH]->(p2)
CREATE (p3)-[:FOLLOWS]->(p1)

// Query: Find friends of Aarav
MATCH (p:Person {name: "Aarav"})-[:FRIENDS_WITH]-(f:Person)
RETURN f.name AS Friend;
```

CYPHER OUTPUT:

```
+---------+
| Friend  |
+---------+
| "Riya"  |
+---------+
```

13. E-Commerce Graph Database

Model `Customer`, `Order`, `Product`, and `Category` nodes. Query products purchased by a customer.

CYPHER SCRIPT:

```
CREATE (cust:Customer {id: "CUST_01", name: "Siddharth"})
CREATE (prod:Product {name: "Logitech MX Master 3", price: 8999})
CREATE (ord:Order {order_id: "ORD_991"})

CREATE (cust)-[:PLACED]->(ord)
CREATE (ord)-[:CONTAINS]->(prod)

// Find all products purchased by Siddharth
MATCH (c:Customer {name: "Siddharth"})-[:PLACED]->()-[:CONTAINS]->(p:Product)
RETURN p.name, p.price;
```

CYPHER OUTPUT:

```
+------------------------+---------+
| p.name                 | p.price |
+------------------------+---------+
| "Logitech MX Master 3" | 8999    |
+------------------------+---------+
```

14. ACID Transactions & Graph Consistency in Neo4j

Demonstrate graph transactional integrity (atomic create/match/set) ensuring no dangling relationships exist.

CYPHER SCRIPT:

```
:auto BEGIN
MATCH (a:Person {name: "Aarav"}), (b:Person {name: "Riya"})
CREATE (a)-[r:WORKS_WITH {since: 2026}]->(b)
SET a.updated_at = timestamp()
COMMIT;
```

EXECUTION SUMMARY:

```
Transaction Committed. Neo4j guarantees full ACID compliance: write locks are acquired on
affected nodes/edges preventing race conditions and ensuring structural graph consistency.
```

15. End-to-End Graph Database Application (Python + Neo4j)

Develop a graph recommendation engine in Python using `neo4j` official driver with CRUD and traversal queries.

PYTHON CODE SNIPPET:

```python
from neo4j import GraphDatabase

driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))

def recommend_products(tx, customer_name):
    query = (
        "MATCH (c:Customer {name: $name})-[:PLACED]->()-[:CONTAINS]->(p:Product)<-[:CONTAINS]->()-
[:CONTAINS]->(rec:Product) "
        "WHERE NOT (c)-[:PLACED]->()-[:CONTAINS]->(rec) "
        "RETURN rec.name AS Recommendation, COUNT(*) AS Frequency ORDER BY Frequency DESC LIMIT 3"
    )
    result = tx.run(query, name=customer_name)
    return [record["Recommendation"] for record in result]

with driver.session() as session:
    recs = session.execute_read(recommend_products, "Siddharth")
    print("Recommended Products:", recs)
```

APPLICATION OUTPUT:

```
Recommended Products: ['Mechanical Keyboard', 'Ergonomic Mousepad', 'USB-C Cable']
System Features: Index-free adjacency ensures O(1) traversal time regardless of dataset size.
```